Relazione Parallel Computing

Traccia: Password decryption

Youness Aharram 7198778

Abstract

Questo lavoro affronta il problema della decifratura di password cifrate tramite algoritmo DES, formulato come un attacco a forza bruta all'interno di uno spazio combinatorio di grandi dimensioni basato su date temporali. L'obiettivo è valutare l'efficacia della parallelizzazione confrontando un' implementazione puramente sequenziale con una versione parallela. Lo studio ha compreso l'analisi di diverse politiche di bilanciamento del carico (scheduling statico e dinamico), la variazione del chunk size e il trade-off tra potenza di calcolo e overhead di comunicazione inter-processo. Sono stati inoltre confrontati due modelli architetturali principali: un design di tipo *Shared-Nothing* e uno basato su memoria condivisa con meccanismo di *Global Early Exit*. Il sistema parallelo ottimizzato ha dimostrato un'elevata scalabilità raggiungendo uno speedup massimo di 6.88× rispetto alla versione sequenziale.

Introduzione

La decifratura di password sottoposte ad hashing rappresenta un problema crittografico caratterizzato da un elevato costo computazionale. La motivazione principale di questo progetto è analizzare strategie di parallelizzazione applicabili ad attacchi a forza bruta e confrontarli con le prestazioni ottenibili da una soluzione sequenziale. Dal punto di vista computazionale, esplorare l'intero spazio delle password di 8 caratteri appartenenti all'alfabeto standard `[a-zA-Z0-9./]` risulta impossibile in tempi ragionevoli, in quanto il numero di combinazioni è pari a 2^{48} possibili valori. Per rendere il problema trattabile e misurabile, lo spazio di ricerca è stato ristretto a un sottoinsieme realistico costituito dalle date nel formato `YYYYMMDD`, riducendo drasticamente il numero di combinazioni da testare. Partendo da queste premesse, il progetto si pone diversi obiettivi. Il primo passo è l'implementazione pratica: verrà creata un'architettura parallela, ed una versione puramente sequenziale che servirà per effettuare i confronti. Successivamente, si passerà alla misurazione delle prestazioni reali del sistema, calcolando lo *Strong Scaling*, per osservare quanto il programma diventa più veloce all'aumentare del numero di core, il *Weak Scaling*, per verificare se il sistema mantiene la propria efficienza quando il carico di

lavoro cresce proporzionalmente al numero di core, e il *Throughput*, per quantificare la potenza netta di decifratura in termini di *Hash Rate*. Infine, l'ultimo obiettivo è analizzare i classici colli di bottiglia della programmazione parallela. Verrà studiata la dimensione ottimale dei pacchetti di lavoro *(chunk)* oltre a misurare l'overhead dovuto alla comunicazione tra processi *(IPC)*.

Implementazione sequenziale

L'implementazione sequenziale costituisce la baseline dell'intero progetto. La valutazione delle sue prestazioni è fondamentale, poiché fornisce il tempo di riferimento assoluto necessario per il calcolo dello *speedup*. L'algoritmo riceve in input una lista di hash DES da decifrare, il relativo *salt* crittografico e un intervallo di anni su cui effettuare la ricerca. Lo spazio delle possibili password è limitato alle date valide nel formato `YYYYMMDD`, generate dinamicamente per ciascun anno dell'intervallo considerato. Dal punto di vista operativo, l'algoritmo itera sequenzialmente sugli anni e, per ogni anno, genera tutte le date valide tramite un generatore incrementale. Questo approccio evita la materializzazione completa dello spazio di ricerca in memoria, riducendo l'overhead e migliorando l'efficienza. Ogni data candidata viene sottoposta alla funzione di hashing DES e il valore ottenuto viene confrontato con l'insieme degli hash target. Per ottimizzare le operazioni di lookup, gli hash da decifrare sono memorizzati in una struttura dati di tipo *set*, che garantisce un tempo medio di accesso costante. Inoltre, viene mantenuta una struttura di risultati che associa ciascun hash alla password trovata. Quando viene identificata una corrispondenza, l'hash viene rimosso dal set dei rimanenti, riducendo progressivamente lo spazio di confronto. L'algoritmo implementa una strategia di terminazione anticipata. Se tutti gli hash vengono risolti prima dell'esplorazione completa dello spazio di ricerca, l'esecuzione viene interrotta immediatamente. Questa ottimizzazione è particolarmente efficace in ambito sequenziale, dove non introduce alcun overhead di sincronizzazione. Infine, durante l'esecuzione viene mantenuto un contatore del numero totale di tentativi effettuati, utile per il calcolo del *throughput* (*hash rate*), insieme al tempo totale di esecuzione, necessario per le successive analisi prestazionali.

Identificazione delle parti parallelizzabili

L'analisi del flusso di esecuzione della versione sequenziale evidenzia una proprietà fondamentale del problema: la decifratura delle password può essere classificata come un caso di problema *imbarazzantemente parallelo*. In questo contesto, ogni operazione di calcolo risulta indipendente dalle altre. In particolare, la generazione delle date, il calcolo dell'hash DES e il confronto con gli hash target relativi a un determinato anno non dipendono in alcun modo dai risultati ottenuti per anni differenti. Non sono presenti stati condivisi né dipendenze tra iterazioni successive che richiedano sincronizzazione o comunicazione intermedia per garantire la correttezza dell'algoritmo. Di conseguenza, il ciclo più esterno,

quello che itera sull'intervallo degli anni, rappresenta il punto naturale di parallelizzazione. L'intero spazio di ricerca può essere suddiviso in sottoinsiemi indipendenti, assegnando a ciascun processo un gruppo distinto di anni da analizzare. Questa suddivisione in unità di lavoro (*chunk*) consente di distribuire il carico computazionale su più core, mantenendo una struttura semplice e con overhead minimo. Ogni processo opera infatti su una porzione disgiunta dello spazio di ricerca, senza necessità di comunicazione durante la fase di calcolo. Tale proprietà strutturale costituisce la base teorica per l'adozione di un modello di parallelizzazione di tipo *Map-Reduce*: nella fase di *map*, ogni processo esplora il proprio sottoinsieme di anni e produce eventuali corrispondenze tra password e hash; nella fase di *reduce*, i risultati parziali vengono aggregati per ottenere la soluzione finale.

Implementazione parallela

L'implementazione parallela del sistema si basa su un modello di tipo *Map-Reduce*, sfruttando il modulo `multiprocessing` di Python. L'obiettivo è distribuire il carico computazionale su più processi, mantenendo al contempo una struttura semplice e modulare.

Architettura generale

L'algoritmo parallelo mantiene la stessa logica della versione sequenziale, ma suddivide lo spazio di ricerca in unità indipendenti. In particolare, ogni task corrisponde all'elaborazione di un singolo anno. L'intervallo temporale viene quindi trasformato in una lista di task, ciascuno dei quali viene assegnato a un processo del pool. La funzione principale `run_parallel_decrypt` si occupa di:

- generare la lista dei task (uno per anno),
- configurare il grado di parallelismo (numero di core),
- definire la strategia di scheduling,
- avviare l'esecuzione parallela,
- aggregare i risultati parziali (fase di *reduce*).

Fase di Map

La fase di *map* è implementata dalla funzione worker `decrypt_single_year`. Ogni processo:

- riceve in input un anno, la lista degli hash target e il *salt*,
- genera tutte le date valide per quell'anno,
- calcola l'hash DES di ciascuna data,

- confronta il risultato con gli hash target locali,
- mantiene un contatore dei tentativi effettuati,
- registra informazioni di profiling (PID, tempo di inizio e fine).

Per ridurre l'overhead di comunicazione, ogni worker opera su una copia locale degli hash target (`set`), evitando accessi concorrenti a strutture condivise durante la fase di calcolo.

Fase di Reduce

Al termine dell'esecuzione parallela, i risultati parziali prodotti dai worker vengono raccolti e combinati. Questa fase avviene nel processo principale in cui viene fatta un aggregazione del numero totale di tentativi e la fusione dei risultati parziali in una struttura globale.

Strategie di scheduling

- **Static scheduling**: il carico di lavoro viene suddiviso in blocchi di dimensione fissa. Ogni processo riceve un insieme predefinito di task. Questa strategia riduce l'overhead di scheduling ma può portare a squilibri di carico.
- **Dynamic scheduling**: i task vengono assegnati dinamicamente. I processi richiedono nuovi task non appena terminano quelli assegnati. Questo approccio migliora il bilanciamento del carico, ma può introduce un overhead maggiore dovuto alla gestione più frequente delle assegnazioni.

Gestione del chunk size

La scelta della dimensione dei *chunk* rappresenta un aspetto critico. Una granularità troppo fine migliora il bilanciamento ma aumenta l'overhead di scheduling; al contrario, una granularità più grossolana riduce l'overhead ma può causare inattività di alcuni processi. Nel caso specifico, un singolo anno costituisce un buon compromesso tra costo computazionale e semplicità di suddivisione.

Variante con stato condiviso (Global Early Exit)

È stata inoltre implementata una variante alternativa, `run_decrypt_global_exit`, che introduce un meccanismo di terminazione anticipata globale tramite memoria condivisa (`multiprocessing.Manager`). In questa versione:

- gli hash rimanenti sono memorizzati in una struttura condivisa (`manager.dict`),
- ogni processo verifica ad ogni iterazione se esistono ancora hash da decifrare,
- quando un hash viene trovato, viene rimosso globalmente.

Questa soluzione consente un *early exit* globale, ma introduce significativi costi di sincronizzazione:

- accessi concorrenti alla struttura condivisa,
- overhead di serializzazione,
- utilizzo implicito di lock.

Risultati e Analisi

Setup Sperimentale

Gli esperimenti sono stati condotti con l'obiettivo di confrontare l'implementazione puramente sequenziale con quella parallela. Le misurazioni sono state effettuate variando tre parametri fondamentali:

- Il numero di processi *worker* impiegati, scalando da 1 fino a un massimo di 20 core logici.
- La dimensione dello spazio di ricerca (da 2 a 256 anni solari), per osservare la risposta del sistema all'aumentare del carico.
- La politica di scheduling dei task, confrontando l'allocazione *static* con quella *dynamic*.

Confronto prestazioni tra sequenziale e parallelo

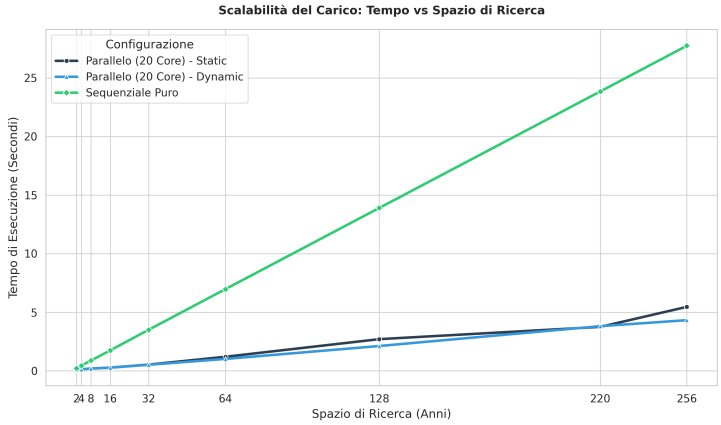

Figure 1: Confronto performance temporali tra la versione sequenziale e parallela

Dalla figura 3 si osserva che il tempo di esecuzione dell'algoritmo sequenziale cresce linearmente all'aumentare dello spazio di ricerca, comportamento coerente per un approccio di tipo *brute-force*. L'implementazione parallela mostra una drastica e significativa riduzione dei tempi assoluti. Testando il carico massimo (*spazio di ricerca* pari a 256), il tempo passa da 27.75s della versione sequenziale a soli 4.32s sfruttando l'infrastruttura parallela a 20 processi con scheduling dinamico. Anche su carichi intermedi, la parallelizzazione abbatte il tempo da 13.90s a circa 2.11s. Analizzando invece i problemi di dimensione ridotta, il guadagno temporale risulta molto più limitato. In questi scenari, il tempo di esecuzione è dominato dal cosiddetto *overhead di startup*: il tempo impiegato dal sistema operativo per istanziare i nuovi processi e preparare i canali di comunicazione penalizza i benefici del calcolo distribuito.

Speedup e Scalabilità

Per quantificare l'efficienza architetturale è stata utilizzata la metrica dello Speedup (S). Calcolandolo in casi di stress, con 16 core e uno spazio di ricerca pari a 64 anni si ottiene uno speedup di picco pari a:

$$S = \frac{6.97}{1.02} \approx 6.88 \tag{1}$$

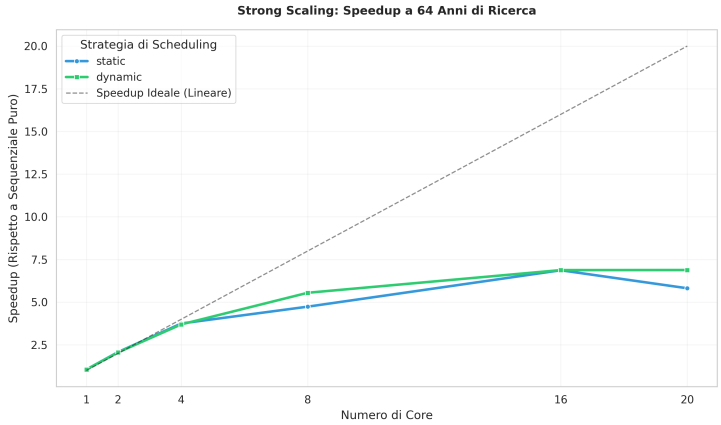

Figure 2: Analisi dello *Speedup* al variare del numero di processi. Si osserva un andamento iniziale quasi lineare, seguito da un progressivo appiattimento asintotico causato dall'overhead di sincronizzazione e dai limiti hardware.

L'andamento dello Speedup rivela una scalabilità iniziale di tipo quasi lineare fino all'impiego di 4 core. Estendendo il grado di parallelismo nella fascia compresa tra i 4 e i 16 processi, il sistema continua a registrare un incremento prestazionale considerevole, seppur con un'efficienza decrescente. Oltre la soglia dei 16 core, la curva entra in una chiara fase di saturazione, dove i benefici apportati da ulteriori unità di calcolo si riducono drasticamente.

Confronto sulle Politiche di Scheduling

Un'evidenza empirica di particolare rilievo emerge dal confronto tra le politiche di scheduling *static* e *dynamic*. Dalla figura 3 si vede che per carichi di lavoro leggeri, la differenza tra le due architetture risulta trascurabile. Tuttavia, sotto sforzo elevato, lo scheduling dinamico si è dimostrato più efficiente. Ad esempio, con uno *spazio di ricerca* pari a 128 e 20 processi attivi, la politica statica ha registrato un tempo di 2.70s, contro i 2.12s di quella dinamica. Questo comportamento si spiega con la natura asincrona dello scheduling *dinamico* che allocando un task alla volta, si previene l'inattività dei core, garantendo un bilanciamento del carico ottimale.

Analisi dello Strong Scaling

Per valutare la capacità di abbattere i tempi di calcolo aggiungendo risorse computazionali, è stata condotta un'analisi di *Strong Scaling*. In questo test, la dimensione del problema globale viene mantenuta costante fissata, mentre si osserva la variazione del tempo di completamento all'aumentare dei processi logici impiegati.

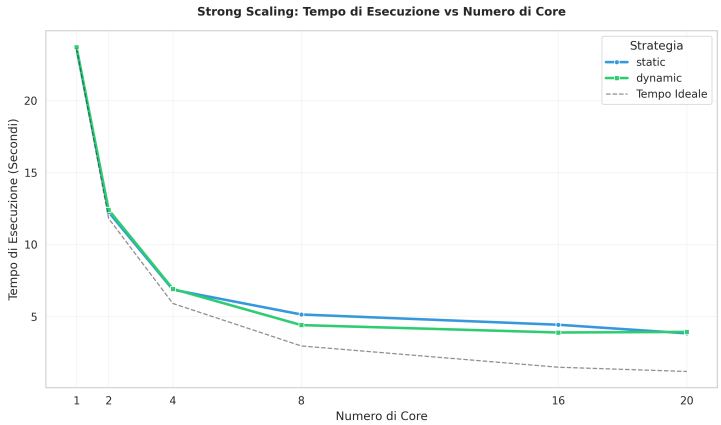

Figure 3: Analisi di *Strong Scaling*: andamento dei tempi di esecuzione al variare del numero di processi a carico di lavoro fisso. La curva evidenzia una scalabilità iniziale quasi ottimale, seguita da un progressivo appiattimento asintotico

Come illustrato in Figura 3, dai dati sperimentali si osserva che il tempo di esecuzione subisce un abbattimento drastico e quasi proporzionale nelle fasi iniziali della parallelizzazione (da 1 a 4 core). In questo primo regime, l'architettura sfrutta in modo ottimale le risorse aggiuntive senza incorrere in penalità di comunicazione. Tuttavia, estendendo ulteriormente il grado di parallelismo, la curva abbandona l'andamento lineare per assumere un comportamento asintotico. Questo fenomeno di saturazione, particolarmente evidente nell'appiattimento

del grafico verso la soglia dei 16 e 20 core. Poiché l'implementazione si basa su un paradigma *Shared-Nothing* privo di lock, tale limite prestazionale è dovuto all'overhead generato dalla comunicazione inter-processo.

Weak Scaling

L'analisi di *Weak Scaling* permette di valutare la capacità del sistema di gestire carichi di lavoro crescenti in modo proporzionale alle risorse computazionali impiegate. In questo scenario, la dimensione del problema per singolo processo viene mantenuta costante (fissata a 10 anni di ricerca per ogni core), con l'obiettivo teorico di mantenere un tempo di esecuzione invariato all'aumentare del numero di core.

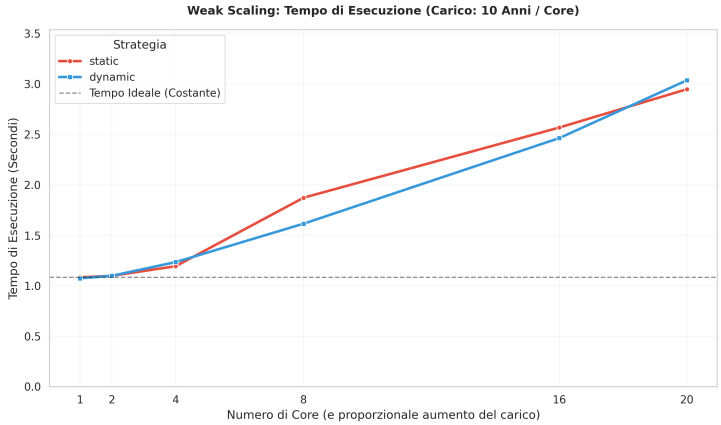

Figure 4: Analisi di *Weak Scaling*: tempo di esecuzione al variare del numero di processi con carico costante per core (10 anni/processo). L'allontanamento dall'andamento orizzontale ideale evidenzia l'incidenza dell'overhead di gestione e della saturazione delle risorse fisiche.

Dai risultati sperimentali riportati in Figura 4, si osserva che il tempo di esecuzione non rimane costante come previsto dal modello ideale, ma tende a crescere progressivamente. Se nel passaggio da 1 a 2 processi l'incremento è quasi trascurabile (da 1.07s a 1.10s, con un'efficienza prossima al 98%), oltre la soglia dei 4 processi il sistema mostra i primi segnali di degradazione prestazionale. L'aumento dei tempi diventa particolarmente marcato a partire dagli 8 processi, dove il tempo di completamento sale a circa 1.62s nel caso della schedulazione dinamica, per poi arrivare in valori superiori ai 3.0s con l'impiego di 20 core. Questo scostamento dalla linearità è riconducibile a due fattori principali inerenti all'architettura dei sistemi paralleli:

- **Costo di orchestrazione:** All'aumentare del numero di processi, il processo *Master* deve gestire un volume di messaggi IPC via via superiore.

- **Saturazione delle risorse hardware:** Nonostante i processi siano indipendenti dal punto di vista software (*Shared-Nothing*), l'aumento dei core operanti comporta comunque una contesa di alcune risorse come il bus di memoria.

Analisi del chunk size

L'ultimo parametro analizzato riguarda la granularità della distribuzione dei task, ovvero il *chunk size*. In un'architettura basata su code dinamiche, il *chunk size* definisce il numero di task (anni di ricerca) inviati a ciascun processo *worker* in un singolo messaggio IPC.

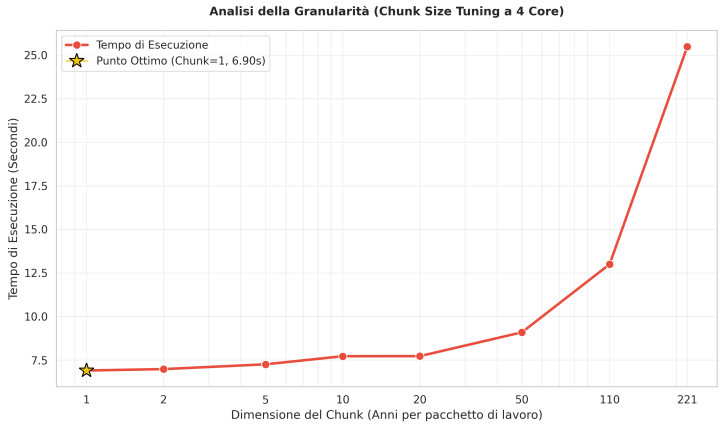

Figure 5: Andamento del tempo di esecuzione al variare della dimensione del *chunk*. Si osserva un degrado prestazionale critico all'aumentare della granularità dei task.

I risultati sperimentali riportati in Figura 5 evidenziano un fenomeno di particolare rilievo: il tempo di esecuzione cresce in modo quasi esponenziale al crescere della dimensione del *chunk*.

- Con un **chunk size unitario (1)**, il sistema ottiene le migliori prestazioni (6.90s), beneficiando di un bilanciamento del carico perfetto dove nessun core rimane inattivo.
- Con **chunk intermedi (10–50)**, il tempo inizia a salire gradualmente (7.72s – 9.09s), segno che la rigidità nell'assegnazione dei blocchi inizia a creare piccoli sbilanciamenti.
- Con un **chunk massimo (221)**, il tempo di esecuzione degenera fino a 25.48s, un valore prossimo a quello della versione sequenziale.

Questo degrado è causato dal cosiddetto *effetto coda*. Quando il *chunk size* è troppo elevato rispetto al numero totale di task, il sistema perde la sua natura

dinamica: i processi ricevono blocchi massicci e indivisibili. Se un processo termina il proprio blocco in anticipo, non può aiutare i processi rimanenti che sono ancora impegnati su pacchetti di lavoro di grandi dimensioni. Di fatto, un *chunk size* eccessivo trasforma l'architettura dinamica in uno scheduling statico, dove i core terminano il lavoro in tempi diversi lasciando la CPU parzialmente inattiva nelle fasi finali del calcolo. I dati dimostrano quindi che, per questo specifico dominio applicativo, l'overhead introdotto da una comunicazione frequente (*chunk*=1) è ampiamente compensato dall'efficienza derivante da un perfetto bilanciamento del carico.

Confronto Architetturale: Shared-Nothing vs Global Exit

Nella progettazione di algoritmi paralleli, uno dei compromessi più critici riguarda la gestione dello stato globale e la comunicazione tra i processi. Per dimostrare l'impatto di queste scelte progettuali, è stato condotto un confronto prestazionale diretto tra:

- **Architettura Shared-Nothing:** In questo modello, i processi *worker* operano in totale isolamento. Ricevono il proprio carico di lavoro e lo elaborano senza mai leggere o scrivere su variabili condivise. La comunicazione con il processo *Master* avviene unicamente al termine del task assegnato, per restituire il risultato.
- **Architettura Global Exit (IPC Centralizzato):** Questo modello introduce una struttura dati condivisa (implementata tramite il costrutto *Manager.dict* di Python) accessibile a tutti i *worker*. Lo scopo è permettere a un processo che individua la password di impostare un *flag* globale. Gli altri processi, interrogando costantemente questa variabile, possono interrompere istantaneamente il proprio lavoro (*Early Exit*), evitando calcoli ridondanti.

Il Paradosso dell'Overhead di Sincronizzazione

Da un punto di vista teorico, l'architettura *Global Exit* dovrebbe garantire tempi di completamento inferiori, poiché minimizza lo spreco di cicli di CPU in caso di ritrovamento anticipato. Tuttavia, i risultati sperimentali (illustrati in Figura 6) smentiscono questa intuizione, rivelando un drastico degrado delle prestazioni. Sottoponendo il sistema al carico massimo (256 anni di ricerca), l'architettura *Shared-Nothing* guidata da uno scheduling dinamico conclude la computazione in soli 4.32s impiegando 20 core. Al contrario, sulla medesima configurazione, l'architettura *Global Exit* impiega ben 9.40s, un tempo più che raddoppiato.

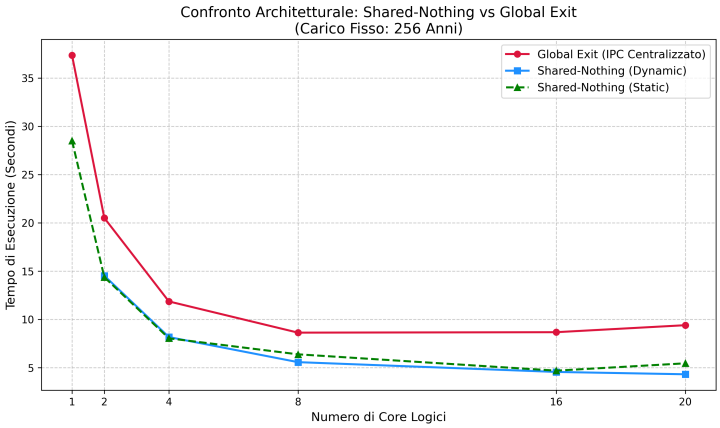

Figure 6: Confronto dei tempi di esecuzione tra le architetture Shared-Nothing e Global Exit a carico massimo fisso (256 anni). Si osserva come la variante centralizzata (linea rossa) subisca un grave degrado prestazionale all'aumentare dei core logici.

Questo divario è la manifestazione di un noto collo di bottiglia nei sistemi distribuiti ovvero il costo di sincronizzazione. Nell'approccio *Global Exit*, l'interrogazione continua della variabile condivisa non è una semplice lettura in RAM. Poiché i processi risiedono in spazi di memoria separati, ogni accesso alla struttura gestita dal *Manager* si traduce in un messaggio che viaggia attraverso i canali di Inter-Process Communication (IPC) del sistema operativo, richiedendo operazioni di serializzazione e deserializzazione. Inoltre, per prevenire *race conditions*, il sistema operativo applica un *Lock* (mutua esclusione) sulla variabile condivisa. A regimi di parallelismo elevati (16 e 20 core), decine di processi eseguono milioni di iterazioni crittografiche al secondo, tentando simultaneamente di accedere al medesimo Lock. Questo genera un traffico IPC tale da saturare il bus di sistema: i processi trascorrono più tempo in stato di blocco, in attesa di poter interrogare il gestore della memoria, di quanto ne impieghino per calcolare gli hash.

Scalabilità dell'Overhead

Per evidenziare come l'incidenza di questo collo di bottiglia vari in funzione del carico, è stato isolato il comportamento a 20 core per diverse dimensioni dello spazio di ricerca (Figura 7).

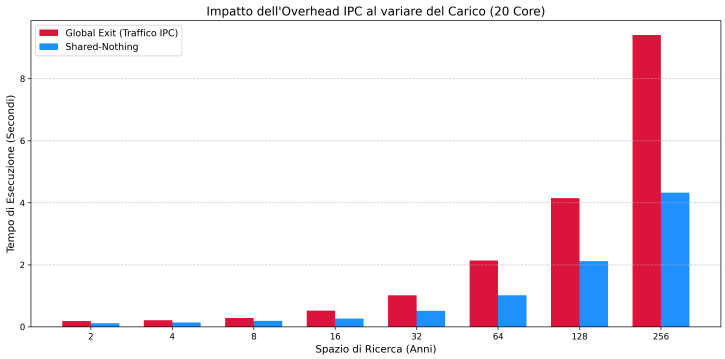

Figure 7: Impatto dell'overhead IPC al variare dello spazio di ricerca, mantenendo fisso l'impiego a 20 core. Il divario assoluto tra le due architetture si amplifica proporzionalmente al volume di calcolo.

Come si evince dall'istogramma, la forbice prestazionale tra i due modelli non è statica, ma si allarga proporzionalmente alla durata dell'esecuzione. Per carichi di lavoro minimi (es. 2 o 4 anni), il volume di traffico IPC non fa in tempo a congestionare il sistema, e le due architetture producono risultati comparabili. Tuttavia, man mano che il problema scala, l'accumulo della latenza dovuta alla contesa del Lock fa esplodere i tempi dell'architettura centralizzata.

Analisi del Throughput

Per valutare la capacità del sistema di erogare potenza computazionale al netto dei tempi di esecuzione assoluti, la metrica di valutazione è stata spostata sul *Throughput*, definito come il numero di iterazioni crittografiche completate nell'unità di tempo (*Hashes per Second*, H/s). In questa fase, il benchmark ha messo a confronto l'esecuzione puramente sequenziale con le varianti parallele *Shared-Nothing* e *Global-Exit*, variando il numero di core da 1 a 20.

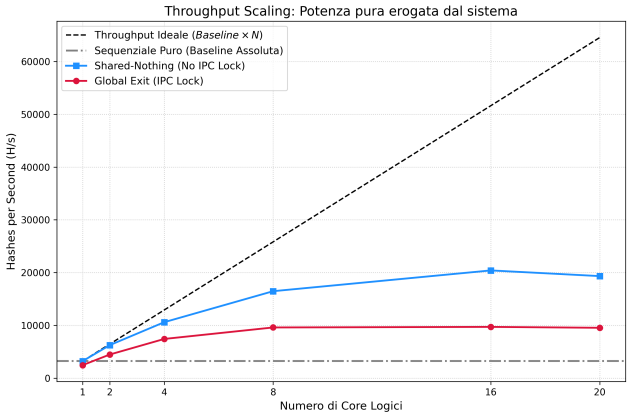

Figure 8: Andamento del Throughput (H/s) al variare dei processi attivi. Si evidenzia il crollo prestazionale dell'architettura centralizzata (Global-Exit) rispetto allo scaling efficiente della variante Shared-Nothing.

Come illustrato in Figura 8, la baseline sequenziale pura si attesta a circa 3.227 H/s. Osservando le due architetture parallele, emergono divergenze prestazionali critiche:

- **Costo di Astrazione Iniziale:** A 1 core logico, l'architettura *Shared-Nothing* (3.209 H/s) si posiziona come il sequenziale puro, pagando un overhead strutturale marginale. Al contrario, l'architettura *Global-Exit* a singolo core crolla immediatamente a 2.452 H/s, evidenziando il costo intrinseco pagato per istanziare e interrogare le variabili di stato condivise ancor prima di introdurre concorrenza.

- **Scalabilità vs Saturazione:** All'aumentare dei processi, la variante *Shared-Nothing* scala in modo estremamente efficiente, raggiungendo un picco di 20.405 H/s a 16 core. La variante *Global-Exit*, invece, subisce un rapido appiattimento: a partire da 8 core, il throughput non riesce a superare la soglia dei 10.000 H/s. Il divario prestazionale a 16 core diventa imponente: l'approccio Shared-Nothing eroga oltre il doppio della potenza computazionale netta. L'aggiunta di ulteriori processi (fino a 20 core) non porta benefici a nessuna delle due architetture.

Per comprendere meglio la natura del crollo dell'architettura *Global-Exit*, è stato concepito un esperimento specifico, progettato per favorire teoricamente proprio l'approccio a memoria condivisa. L'esperimento ha previsto uno spazio di ricerca molto ampio (dal 1800 al 2020), ma con un set di password target deliberatamente concentrate nei primi anni dell'intervallo ($1815 - 1830$). In questo scenario, una terminazione anticipata globale (*Global Early Exit*) dovrebbe garantire un vantaggio cronometrico netto, segnalando a tutti i processi di inter-

rompere l'esplorazione degli anni inutili successivi al 1830. I risultati empirici registrati con l'utilizzo di 4 processi logici hanno restituito le seguenti metriche:

- **Global-Exit:** $79,571$ calcoli eseguiti in 9.97 secondi.
- **Shared-Nothing:** 80.719 calcoli eseguiti in 6.72 secondi.

L'architettura centralizzata ha funzionato correttamente dal punto di vista logico: ha bloccato i core inutili risparmiando **1.148** computazioni crittografiche (circa l' **1.4%** del totale). Tuttavia, dal punto di vista cronometrico, l'esecuzione è risultata **1.5 volte più lenta** della versione *Shared-Nothing*. Questo test dimostra come nell'ambito di algoritmi ad altissima frequenza (come la decrittazione *brute-force*), l'overhead generato dalla comunicazione inter-processo, dall'acquisizione dei Lock e dalla sincronizzazione delle strutture dati condivise supera di ordini di grandezza il tempo richiesto per l'hashing crittografico puro.

Conclusioni

Il presente lavoro ha analizzato e profilato le prestazioni di un algoritmo crittografico di forza bruta parallelizzato, confrontando diverse strategie di allocazione del carico e architetture di gestione della memoria. L'analisi sperimentale ha confermato che, pur trattandosi di un problema intrinsecamente parallelizzabile, l'efficienza globale del sistema è strettamente vincolata dai costi di orchestrazione infrastrutturale. I test di scalabilità (*Strong* e *Weak Scaling*) hanno evidenziato come l'aggiunta di risorse computazionali garantisca incrementi quasi lineari solo fino a regimi di parallelismo medi. Oltre la soglia dei 16-20 processi, il sistema va in saturazione a causa del limite fisico del hardware e del crescente traffico IPC. Il risultato architetturale più rilevante emerso dalla profilazione è la netta superiorità del modello *Shared-Nothing* rispetto all'approccio a memoria condivisa (*Global-Exit*). Si è dimostrato che, in task computazionali ad altissima frequenza, l'overhead generato dall'acquisizione dei lock per la sincronizzazione dello stato globale supera di gran lunga il costo crittografico stesso. In questi scenari, isolare i processi tollerando una parziale ridondanza di calcolo risulta una strategia ingegneristica molto più efficiente rispetto al tentativo di forzare un'uscita anticipata condivisa. Sulla base dei dati di *throughput* raccolti, la configurazione ottimale per massimizzare le prestazioni sull'hardware a disposizione risulta essere:

- Impiego di un numero di core pari a 16.
- Architettura rigorosamente *Shared-Nothing*.